



School of Information Technology, Engineering, Mathematics and Physics

CS341 – Software Quality Assurance and Testing

Semester 2, 2026

Assignment 1 – Software Testing

This assignment covers both the theoretical and the practical aspects of this course. It combines unit testing with JUnit 5 and JaCoCo, test design using Boundary Value Analysis and Equivalence Partitioning, defect logging, and the evaluation of a mobile application into one deliverable. The marking rubric is in liaison with the Course Outline and the Bachelor of Software Engineering program map. Rubrics have been taken from ACS-SCIMS rubrics V1.0.

Learning Outcomes

This assignment tests the following course learning outcomes and the associated CBOK attributes:

- CLO 1 – Appraise verification and validation techniques in the various phases of software development life cycles.
- CLO 3 – Build test designs using equivalence partitioning, boundary value analysis and state transition techniques.
- CLO 4 – Evaluate the different tools available for automated testing.

Assignment Details

Item	Detail
Due Date	Friday 11 September 2026 @ 11:55 pm (Fiji Standard Time)
Submission	Moodle – Assignment 1 Drop Box
Group	Groups of up to 2 students (maximum 2 per group). Only 1 member to make submission.
Weight	15% of the course assessment
Total Marks	80 marks
Late Penalty	10% of the awarded marks per day, without prior notification and permission from the Course Coordinator.

Note: The marking rubric on the final page will be used to assess this assignment. Individual tasks also carry their own mark allocations.

Section A – Unit Testing with JUnit 5 and JaCoCo (50 marks)

Objective

The objective of this section is to test your ability to write and execute unit tests using JUnit 5, and to measure test coverage using the JaCoCo plugin in a Maven-based Java project.

Scenario

You will be provided with a running Maven-based Java project implementing a simple Employee Performance Rating System. Each employee has a name and a performance score (0–100). Based on the score, the system returns a performance rating and a bonus eligibility status.

Rating bands:

- Outstanding: 85 and above
- Exceeds Expectations: 70–84
- Meets Expectations: 50–69
- Needs Improvement: below 50

Bonus eligibility:

- An employee is bonus eligible if the score is 50 or above; otherwise, the employee is not bonus eligible.

Tasks

Task 1 – Write JUnit test cases for the Employee class (20 marks)

Your test class must include:

- Positive tests for rating calculation and bonus eligibility determination, covering every rating band.
- Boundary Value Analysis (BVA) tests around each band boundary.
- Equivalence Partitioning (EP) tests using one representative value from each valid and invalid partition.
- Negative tests for invalid input (null name, empty name, whitespace-only name, score < 0, score > 100).

Task 2 – Execute the test suite (10 marks)

- Run all test cases using mvn clean test and confirm that the suite passes with no errors.
- Assertions must be precise: assert the exact expected value, not merely that a value is non-null.
- Include a screenshot of the successful Maven test run in your report.

Task 3 – Integrate JaCoCo and generate a coverage report (20 marks)

- Add and configure the JaCoCo Maven plugin in pom.xml.
- Generate a test coverage report. The target is 100% line and branch coverage.
- Include a screenshot of the JaCoCo summary page and commit the generated report with your project.

Hints / Steps

1. Extract the provided project zip file.
2. Write the test code based on the tasks above.
3. Run the tests using: `mvn clean test` or using the IDE.
4. Generate the coverage report using: `mvn verify` or the IDE.
5. Open the JaCoCo report in your browser at: `target/site/jacoco/index.html`

Section B – Test Design, Execution and Defect Logging (30 marks)

Pseudocode

Method: computeOverstayCharge

Description: This method calculates the overstay charge for a vehicle parked in a paid car park, based on the number of hours overstayed, the type of vehicle, and the type of parking permit held.

Rules:

Car (Visitor permit):

- 1–4 hours → \$2.00 per hour
- 5–12 hours → \$4.00 per hour
- 13–48 hours → \$8.00 per hour, and the vehicle is flagged for clamping

Car (Resident permit):

- 1–12 hours → \$1.00 per hour
- 13–48 hours → \$3.00 per hour

Truck (any permit type):

- \$15.00 per hour for every hour

Parameters:

- int hoursOverstayed — valid range 1 to 48 inclusive
- int vehicleType — 1 = Car, 2 = Truck
- int permitType — 1 = Visitor, 2 = Resident

Returns:

- The total charge, or -1 if any input is invalid.

Tasks

Task 4 – Test case identification and Unit Test Plan (10 marks)

- Identify test cases for the given functionality using Boundary Value Analysis (BVA) and Equivalence Partitioning (EP).
- State the valid and invalid equivalence classes for each parameter, and the boundary values you derived from them.
- Document all test cases in a Unit Test Plan (UTP) workbook using the template below.

TC ID	Objective / Partition	Technique	Inputs	Expected Result	Actual Result	Status
TC-01	Lower boundary of valid hours	BVA	0, 1, 1	-1		
TC-02	Car / Visitor, tier 1	EP	3, 1, 1	6.00		

The two rows above are examples only. Replace them with your own complete set of test cases.

Task 5 – Implementation, execution and defect logging (10 marks)

- Write a complete Java program `CalcOverstayCharge.java` implementing `computeOverstayCharge`.
- Execute the test cases from your UTP one by one and compare the actual result with the expected result documented in the plan.
- If the actual result matches the expected result, mark the test case as PASSED in the Result column.
- If the actual result differs from the expected result, mark the test case as FAILED and add an entry to the Defect Tracker workbook.
- The defect description must contain the preconditions (if any), the inputs, the actual output, the expected output, and any other information that will help a developer debug and fix the defect.
- Execute all test cases and record a result for every one of them.

Defect Tracker – required fields:

Field	What to record
Defect ID	Unique identifier, e.g. DEF-01
Related TC ID	The test case that exposed the defect
Summary	One-line statement of the problem
Description	Preconditions, inputs, actual output, expected output, steps to reproduce
Severity / Priority	Critical, Major, Minor, Trivial / High, Medium, Low
Status	Open, In Progress, Fixed, Retested, Closed
Reported By / Date	Group member name and date raised
Resolution	What was changed in the code to fix the defect

Task 6 – Automated unit tests and coverage (10 marks)

- Write automated unit tests in JUnit 5 (`CalcOverstayChargeTest.java`).
- Update the code so that 100% statement, branch and path coverage is achieved.
- Fix all defects logged in Task 5 and re-run the suite until every test passes.
- Update the Status column in your Defect Tracker once each defect has been retested and closed.

Submission Guidelines

Submit the following files to the Assignment 1 Drop Box on Moodle:

- Assignment 1 report (PDF) covering Section A screenshots, and Section B Task 4. State your Group number and members name on the cover page.
- Java project files as a single Zip or Rar archive, containing both the Section A and Section B Maven projects.
- Unit Test Plan workbook (Excel).
- Defect Tracker workbook (Excel).
- ReadMe file describing how to build and run the projects, and listing each group member and their contribution.
- Only 1 member has to make the submission.

File naming convention:

```
CS341_A1_Report_GroupX.pdf  
CS341_A1_Java_GroupX.zip  
CS341_A1_UTP_GroupX.xlsx  
CS341_A1_DefectTracker_GroupX.xlsx
```

Replace X with your group number. Incorrect submission naming will result in a high penalty.

Grading and Academic Integrity

6. This assignment is worth 80 marks and 15% of your course assessment.
7. Late delivery without prior notification and permission from the Course Coordinator will result in a loss of 10% of the marks per day.
8. LATE SUBMISSION BEYOND FIVE DAYS = ZERO (0).
9. Plagiarism and cheating in any form are strictly prohibited. If found, the complete Assignment 1 will be nullified.
10. No two groups should submit the same or similar assignment.
11. Each group must submit a short peer contribution statement in the ReadMe. Marks may be adjusted individually where contribution is clearly uneven.

Selective AI Usage

AI tools are restricted for certain functions in this assessment task. Any use of generative AI must be appropriately acknowledged.

Students are permitted to use generative AI tools (e.g., ChatGPT, Microsoft Copilot, or similar AI systems) to support their learning and research during this assignment. However, AI tools may only be used as supplementary research aids and must not be used to generate the full assignment content.

Acceptable Uses of AI

Students may use generative AI tools for the following purposes:

- ❖ Conducting background research on software testing.
- ❖ Understanding technical terminologies related to software testing.
- ❖ Generating ideas or brainstorming possible approaches to software testing.
- ❖ Identifying relevant case studies or real-world examples of software testing.

Unacceptable Uses of AI

Students must not use generative AI tools to:

- ❖ Generate the entire report or parts of the report.
- ❖ Automatically write answers to the assignment questions.
- ❖ Produce complete sections such as the test analysis, test report generation, etc.
- ❖ Submit AI-generated content as their own work without substantial personal interpretation and analysis.

Queries

- Maintain contact through the Moodle discussion forum named Assignment Discussion Forum.
- Consult the Lecturer / Course Coordinator during the online consultation hours or tutorial hours for queries.

Assignment 1 – Assessment Rubric

CS341 Software Quality Assurance and Testing, Semester 2, 2026 — total 100 marks (20% of course assessment).

Criteria	Marks	Excellent (A, 80–100%)	Good (B, 65–79%)	Satisfactory (C, 50–64%)	Poor (D/F, <50%)
A – Task 1: JUnit Test Design & Coverage of Cases	20	Comprehensive tests: all rating bands, bonus eligibility, full BVA and EP, and all invalid inputs asserted with the correct exceptions.	Covers most rating bands, boundaries and some invalid cases; minor gaps.	Covers core rating bands and bonus eligibility but lacks edge cases or invalid inputs.	Minimal or incomplete tests; major cases missing.
A – Task 2: Test Execution & Correctness	10	All tests run successfully with mvn test, no errors; assertions precise and correct.	Tests run with a few minor issues; assertions mostly correct.	Tests run but with frequent assertion gaps or overlooked checks.	Tests fail to run, or assertions largely incorrect.
A – Task 3: JaCoCo Integration & Coverage	20	JaCoCo fully integrated; 100% line and branch coverage achieved with a valid report.	Coverage $\geq 95\%$; minor gaps in branches; report included.	Coverage $\geq 85\%$; some missing classes or branches; report incomplete.	Coverage $< 85\%$ or no report submitted.
B – Task 4: Test Case Identification & Unit Test Plan	10	Comprehensive, well-documented test cases covering all boundaries and equivalence partitions, with a clear UTP.	Good range of test cases with minor gaps; UTP mostly clear.	Limited test cases with several gaps; UTP lacks detail or clarity.	Very few or incorrect test cases; UTP missing or poorly prepared.
B – Task 5: Implementation, Execution & Defect Logging	10	Fully working program with correct logic; all test cases executed and results logged accurately; defect tracker complete and professional.	Mostly working program with minor errors; test cases executed with some gaps; defect tracker mostly complete.	Program runs with major issues; only some test cases executed; defect tracker incomplete or unclear.	Program incorrect or missing; test execution absent; no defect logging.
B – Task 6: Automated JUnit Testing & Coverage	10	Complete JUnit tests with 100% statement, branch and path coverage; all defects fixed and all tests pass.	Good JUnit tests with high coverage ($> 80\%$); minor defects remain.	Limited JUnit tests with partial coverage (50–70%); some tests fail.	Minimal or no automated testing; very low coverage; many failures.
Total	80				

The End